

Digital Signal Processing

for Intelligent Multi-Sensor Fusion

PHASE 5 — RESEARCH-GRADE NOTES

Trinetra-AI

An Intelligent Multi-Sensor Fusion Framework for GPS-Denied Navigation

Every concept in this document is developed for the sensors of Trinetra-AI — **accelerometer, gyroscope, magnetometer, IMU, GPS, barometer**, and the future **quantum magnetometer** — with the single goal of understanding how DSP *cleans, analyses, transforms, and prepares* raw sensor signals *before* they reach the sensor-fusion filters and AI modules. Intuition first, then the mathematics, then the derivation, then the exact place it lives in the pipeline.

Beginner intuition → formal definitions → derivations → code → project mapping.

Contents

1	Introduction	2
1.1	What is signal processing?	2
1.2	Why signal processing is required before sensor fusion	2
1.3	Why navigation systems depend on signal processing	2
1.4	Where DSP appears across domains	3
2	Fundamentals of Signals	3
2.1	The signal taxonomy	3
2.2	Continuous, discrete, analog, digital	3
2.3	Deterministic vs random	4
2.4	Stationary vs non-stationary	4
2.5	Periodic vs aperiodic	4
2.6	Energy vs power signals	4
2.7	Fundamentals reference table	5
3	Sampling	5
3.1	Core definitions	5
3.2	The Sampling Theorem and aliasing	5
3.3	Oversampling, undersampling, quantisation, ADC, resolution	6
3.4	Sampling across the sensor suite & synchronisation	6
4	Noise	7
4.1	What is noise?	7
4.2	Taxonomy of sensor noise	7
4.3	Noise metrics: SNR, power, density, covariance	7
4.4	How noise affects each sensor	8
4.5	How AI models handle noisy sensor data	8
5	Frequency Domain	8
5.1	Core quantities	9
5.2	Time domain vs frequency domain	9
5.3	Why analyse sensor signals in frequency	9
6	Fourier Transform	9
6.1	From Fourier series to Fourier transform	10
6.2	Discrete Fourier Transform (DFT)	10
6.3	Fast Fourier Transform (FFT)	10
6.4	Practicalities: leakage, windowing, resolution	10

6.5	How FFT helps Trinetra-AI	11
7	Digital Filters	11
7.1	Why filters, and analog vs digital	11
7.2	FIR filters	11
7.3	IIR filters	12
7.4	Linear, adaptive, recursive/non-recursive	12
8	Low-pass Filter	13
9	High-pass Filter	13
10	Band-pass Filter	14
11	Butterworth Filter	14
11.1	Order, cut-off, and comparison with other IIR families	15
12	Moving Average Filter	15
13	Savitzky–Golay Filter	17
14	Advanced Signal Processing	17
14.1	Short-Time Fourier Transform (STFT)	18
14.2	Wavelet Transform (CWT / DWT)	18
14.3	Hilbert transform, cepstrum, EMD	18
14.4	Adaptive & Kalman filtering (bridge to Phase 7)	19
15	Signal Processing Libraries	19
16	Research Papers	19
16.1	Foundational	20
16.2	Recent & navigation-specific (DSP × learning)	20
17	Signal Processing Pipeline inside Trinetra-AI	21
18	Practical Implementation	22
19	Project Mapping	22
20	Interview & Research Preparation	22
20.1	40 interview questions	22
20.2	25 viva questions	24
20.3	20 research questions	24
20.4	15 coding questions	25

21 Glossary, Summary & Roadmap	25
21.1 Glossary	25
21.2 Summary	26
21.3 Roadmap: how Phase 5 connects to Phases 6 & 7	26

1 Introduction

1.1 What is signal processing?

A **signal** is any physical quantity that carries information and varies with an independent variable (usually time). **Signal processing** is the set of mathematical and computational operations that *analyse, modify, and extract information* from signals. **Digital Signal Processing (DSP)** performs these operations on signals that have been *sampled* (discretised in time) and *quantised* (discretised in amplitude), so they can be processed by a computer.

Intuitively, a raw sensor stream is a noisy, cluttered radio broadcast; DSP is the tuner, amplifier, and noise-canceller that turns it into something you can actually listen to. For Trinetra-AI, the “broadcast” is the true motion of the vehicle, and the “static” is sensor noise, vibration, bias, and drift. DSP is what recovers the motion from the static.

1.2 Why signal processing is required before sensor fusion

A Kalman filter (Phase 7) is provably optimal *only* under specific assumptions: zero-mean, white, Gaussian measurement noise with a *known* covariance **R**. Real sensors violate every part of that sentence — they carry coloured noise, vibration harmonics, biases, and outliers. DSP is the stage that *makes the assumptions true enough*: it removes out-of-band vibration, attenuates high-frequency noise, rejects spikes, and — crucially — lets you *measure* the residual noise so **R** and the process noise **Q** can be set honestly.

If you feed a filter garbage, no amount of clever fusion math saves you (*garbage-in, garbage-out*). In Trinetra-AI, DSP sits between raw acquisition and both the AI modules and the fusion core. A poorly filtered gyro signal makes the drift-prediction network learn noise; an unfiltered accelerometer injects vibration energy straight into the position estimate.

1.3 Why navigation systems depend on signal processing

Inertial navigation *integrates* sensor signals over time. Integration is a low-frequency-emphasising operation: it amplifies slow errors (bias, drift) and accumulates them without bound. Because the very act of navigating amplifies low-frequency error, DSP must aggressively characterise and suppress that error before integration. The double integration of acceleration into position means a tiny bias b becomes a position error $\frac{1}{2}bt^2$ — quadratic in time. DSP is the first line of defence.

1.4 Where DSP appears across domains

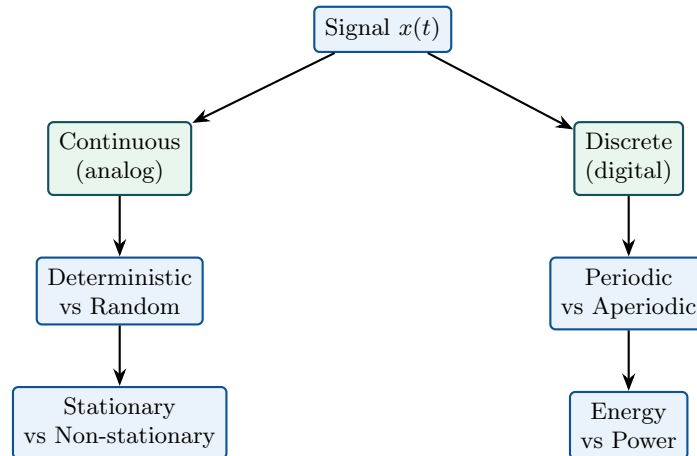
Domain	Signal-processing challenge	Trinetra-AI relevance
Robotics	motor/vibration noise on IMU; encoder aliasing	clean IMU for odometry
Autonomous vehicles	GNSS multipath, road vibration, wheel-slip	band-limit & outlier-reject before fusion
UAVs / drones	strong rotor vibration harmonics on accel/gyro	notch/low-pass at rotor frequency
Aerospace	long integration times, extreme drift discipline	Allan-variance noise budgeting
Quantum navigation	ultra-low-noise field signals, but new noise physics (shot noise)	exploit cleaner streams, model quantum noise
Intelligent sensors	on-chip filtering, edge constraints	lightweight FIR/IIR on Jetson/MCU

Quantum-navigation note. A quantum magnetometer (e.g. optically-pumped or NV-diamond) delivers a magnetic-field time series with far lower drift than a MEMS magnetometer, but with its *own* noise signature (photon *shot noise*, laser-intensity noise). From the software layer it is still a sampled signal that needs the same DSP toolbox — only the noise *model* changes.

2 Fundamentals of Signals

Before processing anything, we must classify what kind of signal each sensor produces. The taxonomy below is not academic pedantry: it dictates which tools are legal (e.g. stationarity governs whether a plain FFT is meaningful, energy/power governs which metrics apply).

2.1 The signal taxonomy



2.2 Continuous, discrete, analog, digital

A **continuous-time signal** $x(t)$ is defined for every real t . A **discrete-time signal** $x[n] = x(nT_s)$ is defined only at integer indices n , where T_s is the sampling period. An **analog signal** is continuous in *both* time and amplitude; a **digital signal** is discrete in *both* (sampled *and* quantised).

The physical world (the true acceleration of the vehicle) is analog. The sensor's ADC produces a digital signal. Every sensor in Trinetra-AI is therefore an analog-to-digital bridge, and the fidelity of that bridge (sampling rate, bit resolution) bounds everything downstream.

2.3 Deterministic vs random

A **deterministic** signal is exactly predictable by a formula, e.g. $x(t) = A \sin(2\pi ft)$. A **random (stochastic)** signal can only be described statistically — by its mean, variance, and probability distribution.

Sensor output is the *sum* of a (partly deterministic) motion signal and a random noise process: $z[n] = s[n] + \eta[n]$. This decomposition is the reason DSP and probability are inseparable in navigation: we filter the deterministic part and statistically characterise the random part (which becomes **R**).

2.4 Stationary vs non-stationary

A random signal is **(weakly) stationary** if its mean, variance, and autocorrelation do not change with time: $\mathbb{E}[x[n]] = \mu$, $\text{Var}(x[n]) = \sigma^2$, $\text{Cov}(x[n], x[n+k]) = \gamma(k) \forall n$. Otherwise it is **non-stationary**.

A stationary IMU signal is what you get from a *static* sensor on a bench — appropriate for noise characterisation (Allan variance, PSD). A *moving* vehicle produces *non-stationary* signals whose frequency content changes constantly; these demand time–frequency tools (STFT, wavelets, §14) rather than a single global FFT.

2.5 Periodic vs aperiodic

$x(t)$ is **periodic** with period T if $x(t+T) = x(t)$ for all t ; the fundamental frequency is $f_0 = 1/T$. Otherwise it is **aperiodic**.

Rotor vibration on a UAV is quasi-periodic at the blade-pass frequency — a periodic nuisance we can target with a notch filter. A one-off pothole impact on a car's accelerometer is aperiodic (transient) — better handled by wavelets or outlier rejection.

2.6 Energy vs power signals

Energy of a discrete signal: $E = \sum_{n=-\infty}^{\infty} |x[n]|^2$. **Average power**: $P = \lim_{N \rightarrow \infty} \frac{1}{2N+1} \sum_{n=-N}^N |x[n]|^2$. An **energy signal** has $0 < E < \infty$ (and $P = 0$) — typically transients. A **power signal** has $0 < P < \infty$ (and $E = \infty$) — typically periodic or stationary-noise signals.

A footstep impact is an energy signal; the continuous vibration hum of a running motor is a power signal. This distinction decides which metric is meaningful: transient events are measured by energy, ongoing noise by power (and hence by *power spectral density*, §5).

2.7 Fundamentals reference table

Property	Test	Sensor example	Consequence in Trinetra-AI
Continuous/discrete	is it sampled?	true accel vs ADC output	sets sampling design (§3)
Deterministic/random	formula vs statistics	motion vs noise term	split filter vs characterise R
Stationary/non-stationary	ADF / rolling stats	static bench vs driving	FFT vs STFT/wavelet
Periodic/asperiodic	autocorrelation peaks	rotor hum vs pothole	notch vs transient detection
Energy/power	E vs P finite	impact vs motor hum	energy vs PSD metric

```
import numpy as np
# Classify a window as (approximately) stationary via rolling-stat variation
def is_stationary(x, k=8, tol=0.25):
    segs = np.array_split(x, k)
    means = [s.mean() for s in segs]; varr = [s.var() for s in segs]
    # low relative spread of segment means & variances => stationary-ish
    return (np.std(means)/(abs(np.mean(means))+1e-9) < tol) and \
           (np.std(varr)/(np.mean(varr)+1e-9) < tol)
```

Interview prompts. Why can a moving-vehicle IMU signal not be analysed with a single global FFT? Give a sensor example of an energy signal and a power signal. Why is stationarity a prerequisite for meaningful Allan-variance analysis?

3 Sampling

Sampling is the bridge from the analog world to the digital pipeline, and getting it wrong corrupts *everything* downstream in a way no later filter can undo.

3.1 Core definitions

Sampling converts $x(t)$ to $x[n] = x(nT_s)$. The **sampling interval** (period) is T_s ; the **sampling rate / frequency** is $f_s = 1/T_s$ (Hz). The **Nyquist frequency** is $f_N = f_s/2$, the highest frequency representable without ambiguity.

3.2 The Sampling Theorem and aliasing

Ideal sampling multiplies $x(t)$ by a Dirac comb $\text{III}_{T_s}(t) = \sum_n \delta(t - nT_s)$. Multiplication in time is convolution in frequency, and the comb's spectrum is another comb of spacing f_s . Hence the sampled spectrum is a *periodic replication* of the original:

$$X_s(f) = f_s \sum_{k=-\infty}^{\infty} X(f - kf_s).$$

If $X(f)$ is band-limited to $|f| < f_{\max}$, the replicas do *not* overlap iff $f_s - f_{\max} > f_{\max}$, i.e.

$$\boxed{f_s > 2f_{\max}} \quad (\text{Nyquist–Shannon}).$$

When $f_s < 2f_{\max}$, replicas overlap and high frequencies *fold* back: a component at $f > f_N$ appears at the alias $f_a = |f - f_s \text{round}(f/f_s)|$.

Aliasing is catastrophic for navigation because a high-frequency vibration (say a 300 Hz rotor tone sampled at 200 Hz) folds into a low-frequency band and is *indistinguishable* from real motion. The gyro drift-prediction network would then “learn” a phantom oscillation.

Cure: an *analog* anti-alias low-pass filter *before* the ADC, plus adequate f_s . No digital filter can remove aliasing after the fact.

3.3 Oversampling, undersampling, quantisation, ADC, resolution

Oversampling: $f_s \gg 2f_{\max}$; eases anti-alias filtering and (with averaging) improves effective resolution. **Undersampling:** $f_s < 2f_{\max}$; causes aliasing (occasionally used deliberately in bandpass sampling). **ADC** (analog-to-digital converter) samples *and* quantises. **Resolution** is the number of bits b ; the quantisation step is $\Delta = \frac{V_{\max} - V_{\min}}{2^b}$.

Modelling rounding error as uniform on $[-\Delta/2, \Delta/2]$ gives variance $\sigma_q^2 = \Delta^2/12$. The signal-to-quantisation-noise ratio of a full-scale sinusoid is

$$\text{SQNR}_{\text{dB}} \approx 6.02b + 1.76,$$

i.e. every extra ADC bit buys ≈ 6 dB. A 16-bit IMU ADC therefore has ≈ 98 dB headroom — usually far below the sensor’s intrinsic noise, which is why MEMS noise (not quantisation) dominates.

3.4 Sampling across the sensor suite & synchronisation

Sensor	Typical f_s	Band of interest	Sampling concern
Accelerometer	100–1000 Hz	0–50 Hz motion	anti-alias rotor/vibration tones
Gyroscope	100–1000 Hz	0–50 Hz	same; high rate aids integration
Magnetometer	10–100 Hz	0–10 Hz	slow; watch motor-induced tones
GPS/GNSS	1–10 Hz	< 1 Hz	very low rate; must be up-interpolated
Barometer	10–100 Hz	< 5 Hz	slow vertical dynamics
Quantum mag.	10–1000 Hz	application-dep.	new noise band (shot noise)

The synchronisation problem. Sensors run on *different clocks* at *different rates*. Fusion requires all measurements on one timeline. Three issues: *timestamp alignment* (which clock stamped this sample?), *clock drift* (two oscillators diverge linearly over time), and *interpolation* (resampling the slow GPS onto the fast IMU grid). Sub-millisecond misalignment at high dynamics produces apparent “sensor error” that is really a timing error — one of the most common silent failures in a fusion stack.

```

import numpy as np
def sync_to_reference(t_ref, t_sensor, x_sensor):
    """Linear-interpolate a sensor stream onto the reference (e.g. IMU) timeline."""
    return np.interp(t_ref, t_sensor, x_sensor)

def estimate_clock_offset(t_a, t_b):
    """Crude constant-offset estimate between two clocks (align on shared events)."""
    return np.median(t_a) - np.median(t_b)

```

4 Noise

Noise is the enemy DSP exists to fight, and *characterising* it (not just removing it) is what lets the fusion filter be tuned correctly.

4.1 What is noise?

Noise is the unwanted random component of a measurement, $z[n] = s[n] + \eta[n]$, where $s[n]$ is the true signal and $\eta[n]$ the noise process. Noise is described *statistically*: by its distribution, power, and spectral shape.

4.2 Taxonomy of sensor noise

Noise type	Nature / spectrum	Where it bites
White noise	flat PSD, uncorrelated in time	baseline of all sensors; sets R
Gaussian noise	normal amplitude distribution	MEMS accel/gyro thermal-mechanical
Thermal (Johnson)	$\propto$ temperature; broadband	analog front-end, magnetometer
Shot noise	discrete-arrival (Poisson)	photodetectors — quantum magnetometer
Quantisation noise	uniform, $\Delta^2/12$	ADC (usually sub-dominant)
Magnetic noise	environmental fields, motors	magnetometer heading errors
Environmental	temperature, vibration, EMI	all sensors, context-dependent
Random-walk noise	integral of white noise; variance $\propto t$	gyro angle random walk (ARW)
Bias (offset)	slowly-varying non-zero mean	accel/gyro — integrates to drift
Drift noise	slow trend (temp, ageing)	the core GPS-denied enemy

4.3 Noise metrics: SNR, power, density, covariance

Signal-to-Noise Ratio: $\text{SNR} = P_{\text{signal}}/P_{\text{noise}}$, in decibels $\text{SNR}_{\text{dB}} = 10 \log_{10}(P_s/P_n)$. **Noise power** is the variance σ^2 of a zero-mean noise. **Noise density** is power per unit bandwidth, e.g. accel in $\mu g/\sqrt{\text{Hz}}$, gyro in $^\circ/\sqrt{\text{hr}}$ (ARW). **Covariance** $\mathbf{R} = \mathbb{E}[\eta\eta^\top]$ generalises variance to the multi-axis / multi-sensor case and is exactly the matrix the Kalman filter needs.

Let $\eta[n]$ be white with variance σ^2 . Its running integral $w[n] = \sum_{k \leq n} \eta[k]$ (what happens when we integrate a gyro rate to an angle) has variance

$$\text{Var}(w[n]) = \sum_{k \leq n} \sigma^2 = n \sigma^2 \propto t.$$

The standard deviation grows as $\sqrt{t}$ — this is *angle random walk*, and it is the fundamental reason unaided inertial heading degrades over time. Integrating a *bias* b instead gives bt (linear), and double-integrating gives $\frac{1}{2}bt^2$ (quadratic). These three growth laws — $\sqrt{t}$, t , t^2 — are the DNA of inertial drift.

4.4 How noise affects each sensor

Sensor	Dominant noise & effect
Accelerometer	white + vibration + bias; bias double-integrates to $\frac{1}{2}bt^2$ position error
Gyroscope	ARW + bias instability; bias integrates to bt heading error (then leaks to position)
Magnetometer	environmental/magnetic + hard/soft-iron; corrupts absolute heading
GPS	multipath outliers + receiver noise; non-Gaussian jumps break KF assumptions
Barometer	thermal + slow weather drift; noisy but drift-bounded altitude
Quantum sensor	shot noise (photon statistics) + laser noise; far lower drift, new spectral shape

4.5 How AI models handle noisy sensor data

Three complementary strategies, all used in Trinetra-AI: (1) **learned denoising** — a CNN/TCN or autoencoder maps noisy→clean, as in Brossard et al. (2020) who denoise gyros with a dilated CNN; (2) **noise-robust training** — augment with synthetic noise so the model generalises; (3) **uncertainty estimation** — the network outputs a variance (TLIO-style) that becomes the filter’s adaptive **R**. DSP and ML are partners here: classical filtering removes the easy, out-of-band noise cheaply; the network handles the structured, in-band, non-linear residual.

```
import numpy as np
def snr_db(signal, noise):
    ps = np.mean(signal**2); pn = np.mean(noise**2)
    return 10*np.log10(ps/(pn+1e-12))

def estimate_R_from_static(z_static):
    """Estimate measurement covariance R from a static (no-motion) recording."""
    return np.cov(z_static.T) # z_static: (N, channels)
```

5 Frequency Domain

Many things that look like formless noise in time become obvious structure in frequency. The frequency domain is where vibration tones, harmonics, and noise colour reveal themselves.

5.1 Core quantities

For a sinusoid $x(t) = A \cos(2\pi ft + \phi)$: **amplitude** A is the peak magnitude, **frequency** f (Hz) the cycles per second, **phase** ϕ the offset, **angular frequency** $\omega = 2\pi f$ (rad/s). **Wavelength** $\lambda = v/f$ (for a wave of speed v). **Harmonics** are integer multiples $2f_0, 3f_0, \dots$ of a fundamental f_0 .

The **spectrum** $X(f)$ gives amplitude & phase per frequency. The **power spectrum** / **power spectral density (PSD)** $S_{xx}(f)$ gives power per unit frequency; by the Wiener–Khinchin theorem it is the Fourier transform of the autocorrelation, $S_{xx}(f) = \mathcal{F}\{\gamma(\tau)\}$. A **spectrogram** is $|\text{STFT}|^2$: how the spectrum evolves over time.

5.2 Time domain vs frequency domain

The *time domain* answers “what is the value now?”; the *frequency domain* answers “what periodic components are present, and how strong?”. They are two views of the same information, linked by the Fourier transform (§6). For a stationary noise process the PSD is the natural description; for a transient the time domain is.

5.3 Why analyse sensor signals in frequency

Application	What the frequency domain reveals
Noise identification	the <i>colour</i> of noise (flat = white, $1/f$ = flicker) $\Rightarrow$ correct model
Vibration analysis	rotor/engine tones as sharp peaks at known frequencies
Fault detection	a new/growing spectral peak signals bearing wear, imbalance, looseness
Sensor health	broadband noise-floor rise = degrading sensor
Navigation	separate <i>motion band</i> (low) from <i>vibration band</i> (high) to design filters
Magnetic anomaly	spatial-frequency content of the field map used for localisation

Trinetra-AI uses frequency analysis twice: *offline*, to design filters and budget noise (choose cut-offs, place notches at rotor tones); and *online*, as **features** — dominant frequency, band power, spectral-floor level feed the fault-detection and sensor-health modules (Phase 4 §8 connects here directly).

6 Fourier Transform

The Fourier transform is the mathematical machine that moves between time and frequency. It underlies filtering, feature extraction, and noise analysis.

6.1 From Fourier series to Fourier transform

A periodic signal decomposes into a **Fourier series** — a sum of harmonically related sinusoids: $x(t) = \sum_{k=-\infty}^{\infty} c_k e^{j2\pi k f_0 t}$, $c_k = \frac{1}{T} \int_T x(t) e^{-j2\pi k f_0 t} dt$. Letting $T \rightarrow \infty$ (aperiodic signal) turns the sum into an integral — the **Fourier transform**:

$$X(f) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi f t} dt, \quad x(t) = \int_{-\infty}^{\infty} X(f) e^{+j2\pi f t} df.$$

6.2 Discrete Fourier Transform (DFT)

For a length- N discrete signal $x[n]$, the **DFT** and its inverse are

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{+j2\pi kn/N},$$

where bin k corresponds to frequency $f_k = kf_s/N$. Direct evaluation costs $O(N^2)$ complex multiplies.

6.3 Fast Fourier Transform (FFT)

Split the DFT into even- and odd-indexed samples. With $W_N = e^{-j2\pi/N}$,

$$X[k] = \underbrace{\sum_m x[2m] W_N^{2mk}}_{E[k] \text{ (DFT of evens)}} + W_N^k \underbrace{\sum_m x[2m+1] W_N^{2mk}}_{O[k] \text{ (DFT of odds)}}.$$

Since $W_N^2 = W_{N/2}$, each half is an $N/2$ -point DFT, giving the butterfly $X[k] = E[k] + W_N^k O[k]$, $X[k+N/2] = E[k] - W_N^k O[k]$. Recursing yields the recurrence $T(N) = 2T(N/2) + O(N)$, so

$$T(N) = O(N \log N).$$

For $N = 1024$ this is $\approx 100\times$ faster than the naive DFT — the difference between real-time and not.

6.4 Practicalities: leakage, windowing, resolution

Frequency resolution is $\Delta f = f_s/N$: longer windows resolve closer tones. A finite window implicitly multiplies by a rectangle, whose spectrum has side-lobes — causing **spectral leakage**. Multiplying the window by a taper (Hann, Hamming, Blackman) suppresses leakage at the cost of slightly wider peaks. This matters when detecting a weak fault tone next to a strong motion component.

```
import numpy as np
def spectrum(x, fs, window='hann'):
```

```

x = np.asarray(x, float); N = len(x)
w = np.hanning(N) if window == 'hann' else np.ones(N)
X = np.fft.rfft(x * w)
freqs = np.fft.rfftfreq(N, d=1.0/fs)
mag = np.abs(X) / (w.sum()/2) # amplitude-corrected
return freqs, mag

# Welch PSD (averaged, lower-variance) via SciPy
from scipy.signal import welch
def psd(x, fs): return welch(x, fs=fs, nperseg=256)

```

6.5 How FFT helps Trinetra-AI

Task	FFT role
Noise removal	identify noise band $\Rightarrow$ set filter cut-off; or zero bins & inverse-FFT
Feature extraction	dominant frequency, band energies feed Phase-4 ML modules
Fault diagnosis	emerging harmonic peaks flag mechanical faults
Frequency analysis	separate motion (low) from vibration (high) bands
Magnetic anomaly	spatial-spectral signature of the field for map matching
Sensor health	rising broadband floor $\Rightarrow$ degradation alarm

Complexity / limits. FFT assumes the signal is *stationary over the window*; for a manoeuvring vehicle use the STFT or wavelets (§14). It also assumes periodic extension (hence leakage) and uniform sampling (hence the need for §3 resampling).

7 Digital Filters

A filter reshapes a signal's spectrum — passing wanted bands, attenuating unwanted ones. Filtering is the workhorse pre-processing step before fusion.

7.1 Why filters, and analog vs digital

	Analog filter	Digital filter
Domain	continuous, hardware (R,L,C)	discrete, software on samples
Pros	no sampling, no aliasing, fast	exact, reproducible, tunable, complex responses
Cons	drift, tolerance, fixed once built	needs ADC, limited by f_s , latency
Use in Trinetra-AI	<i>anti-alias</i> before ADC (essential)	all post-ADC processing

7.2 FIR filters

A **Finite Impulse Response** filter computes a weighted sum of the last $M+1$ inputs only (*non-recursive*):

$$y[n] = \sum_{k=0}^M b_k x[n-k], \quad H(z) = \sum_{k=0}^M b_k z^{-k}.$$

Its impulse response has finite length $M+1$.

Properties. FIR filters are *always stable* (no feedback poles) and can have *exactly linear phase* if the coefficients are symmetric ($b_k = b_{M-k}$) — meaning all frequencies are delayed equally, so

waveform shape is preserved. This is precious in navigation: a linear-phase filter delays the whole IMU signal by a known constant ($M/2$) samples, which the fusion timestamp can compensate exactly. Cost: many taps for a sharp cut-off.

7.3 IIR filters

An **Infinite Impulse Response** filter uses feedback (*recursive*):

$$y[n] = \sum_{k=0}^M b_k x[n-k] - \sum_{k=1}^P a_k y[n-k], \quad H(z) = \frac{\sum_k b_k z^{-k}}{1 + \sum_k a_k z^{-k}} = \frac{B(z)}{A(z)}.$$

Its impulse response is (in principle) infinite.

Properties. IIR filters achieve a given sharpness with far fewer coefficients (efficient — good for edge/MCU), but they can be *unstable* (poles must lie inside the unit circle) and generally have *non-linear phase* (frequency-dependent delay, i.e. phase distortion). Butterworth/Chebyshev/Bessel/Elliptic (§11) are IIR designs. For offline processing, *zero-phase* filtering (`filtfilt`, forward+backward) cancels the phase distortion.

7.4 Linear, adaptive, recursive/non-recursive

Class	Meaning & Trinetra-AI use
Linear	output is a linear function of input (superposition holds); all above are linear
Non-recursive	FIR (no feedback); stable, linear-phase pre-filtering of IMU
Recursive	IIR (feedback); efficient sharp roll-off for band-limiting
Adaptive (LM-S/RLS)	coefficients <i>update online</i> to track changing noise; e.g. cancel rotor vibration whose frequency shifts with throttle

The Least-Mean-Squares filter minimises $\mathbb{E}[e^2]$ where $e[n] = d[n] - \mathbf{w}^T \mathbf{x}[n]$. Stochastic gradient descent on the instantaneous error gives the update

$$\mathbf{w}[n+1] = \mathbf{w}[n] + \mu e[n] \mathbf{x}[n],$$

with step μ controlling convergence vs. stability. This lets Trinetra-AI cancel a *reference* noise (e.g. a tachometer-derived rotor tone) from the IMU adaptively.

```
from scipy.signal import lfilter, filtfilt, firwin
# FIR low-pass (linear phase), applied causally:
b = firwin(numtaps=51, cutoff=20, fs=200) # 20 Hz cut, 200 Hz IMU
y_fir = lfilter(b, [1.0], accel_z)
# IIR zero-phase (offline): use filtfilt to cancel phase distortion
# (b_iir, a_iir designed in Section 11)
```

8 Low-pass Filter

A **low-pass filter (LPF)** passes frequencies below a **cut-off** f_c and attenuates those above it. f_c is conventionally the -3 dB point (half power). The ideal “brick-wall” response is $H(f) = 1$ for $|f| < f_c$, else 0; real filters approximate it with a finite transition band.

A first-order RC-style LPF has transfer function (continuous) $H(s) = \frac{1}{1 + s/\omega_c}$, magnitude $|H(j\omega)| = \frac{1}{\sqrt{1 + (\omega/\omega_c)^2}}$, rolling off at -20 dB/decade. Its discrete exponential form is the EMA (§12).

The LPF is the *most-used* filter in Trinetra-AI. It removes high-frequency vibration and noise from accelerometer/gyro/magnetometer before integration and fusion. **Per sensor:** accel — strip vibration, keep gravity/motion; gyro — remove high-frequency sensor noise before integrating to angle; magnetometer — smooth the slow heading signal; IMU — band-limit to the motion band. **Caution:** too low a cut-off adds lag and eats real dynamics, hurting the fusion estimate. Cut-off is chosen from the FFT-identified motion band (§6).

9 High-pass Filter

A **high-pass filter (HPF)** passes frequencies *above* f_c and attenuates those below — the complement of a LPF. A first-order form is $H(s) = \frac{s/\omega_c}{1 + s/\omega_c}$.

Working & use. The HPF removes slow drift and constant bias (DC), leaving only the dynamic part. In motion detection it isolates rapid changes (steps, impacts, manoeuvres) from the slowly-varying background. In the classic *complementary filter* (§14/Phase 7), the gyro is *high-pass*-fused with the *low-pass* accelerometer: the HPF keeps the gyro’s good short-term dynamics while discarding its slow drift, and the LPF keeps the accel’s good long-term reference while discarding its vibration.

```
from scipy.signal import butter, filtfilt
def highpass(x, fc, fs, order=2):
    b, a = butter(order, fc/(fs/2), btype='high')
    return filtfilt(b, a, x) # zero-phase, offline
# e.g. remove barometer weather drift, keep vertical motion:
baro_motion = highpass(baro_alt, fc=0.1, fs=50)
```

10 Band-pass Filter

A **band-pass filter (BPF)** passes a band $[f_{c1}, f_{c2}]$ and attenuates everything outside it — conceptually a HPF at f_{c1} cascaded with a LPF at f_{c2} . Its complement, a **notch (band-stop)** filter, rejects a narrow band.

Working & applications. A BPF isolates a phenomenon known to live in a specific band: gait analysis (human walking $\approx 1\text{--}3\text{ Hz}$), engine/rotor vibration signatures, or a fault frequency of interest. **Noise removal:** keep only the informative band, discarding both low-frequency drift and high-frequency hiss. **Vibration & fault detection:** band-pass around a bearing’s characteristic frequency to track its energy over time; a *notch* at the rotor tone removes that tone from the IMU while preserving motion.

Sensor example	Band-pass use in Trinetra-AI
Accelerometer	isolate gait band for pedestrian step detection; notch rotor tone on UAV
Gyroscope	extract manoeuvre band; reject narrow vibration lines
Magnetometer	band-limit to the spatial-frequency band of usable anomalies

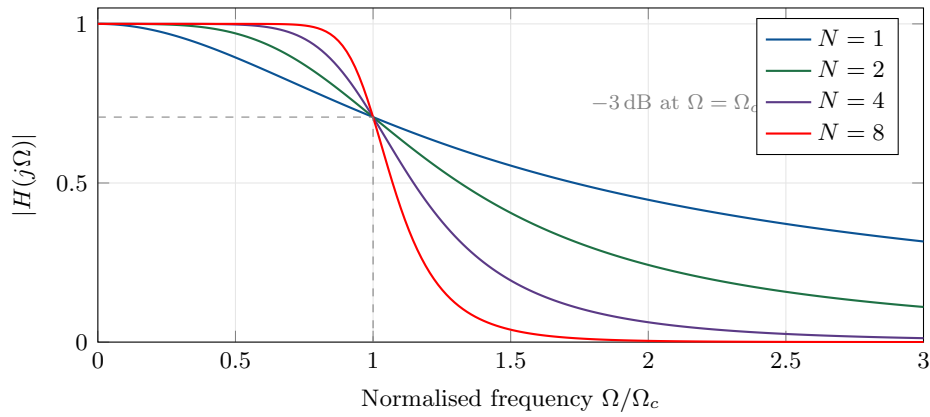
11 Butterworth Filter

The **Butterworth filter** is the IIR filter with a *maximally flat* magnitude response in the passband. Its magnitude-squared response for order N and cut-off Ω_c is

$$|H(j\Omega)|^2 = \frac{1}{1 + (\Omega/\Omega_c)^{2N}}.$$

At $\Omega = 0$, $|H|^2 = 1$; the first $2N-1$ derivatives of $|H|^2$ vanish at the origin — no ripple, the flattest possible passband. At $\Omega = \Omega_c$, $|H|^2 = \frac{1}{2}$ (the -3 dB point) *for every order*. The roll-off is $-20N\text{ dB/decade}$. The poles of $H(s)H(-s)$ solve $1 + (s/j\Omega_c)^{2N} = 0$ and lie equally spaced on a circle of radius Ω_c ; keeping the left-half-plane poles guarantees stability:

$$s_k = \Omega_c \exp\left(j\pi \frac{2k + N - 1}{2N}\right), \quad k = 1, \dots, N.$$



Higher order $\Rightarrow$ sharper transition toward the ideal brick wall, at the cost of more phase distortion and computation.

11.1 Order, cut-off, and comparison with other IIR families

Family	Passband / stopband	Trade-off	When to prefer
Butterworth	flat / flat	smooth, moderate roll-off, mild phase distortion	default sensor smoothing
Chebyshev I	ripple / flat	sharper roll-off, passband ripple	need sharp cut, tolerate ripple
Chebyshev II	flat / ripple	sharp, stopband ripple	flat passband + sharp cut
Elliptic (Cauer)	ripple / ripple	sharpest for given order	tight band constraints
Bessel	flat, linear phase	gentle roll-off, best phase	preserve waveform shape/timing

Butterworth is the Trinetra-AI default for band-limiting IMU/magnetometer signals: its flat passband does not distort the motion band, and its smooth roll-off avoids the ringing that ripple filters add to transient-rich navigation data. Where *timing* must be preserved exactly (e.g. aligning IMU with GPS), a **Bessel** filter's linear phase is preferable, or apply Butterworth with `filtfilt` for zero phase offline.

```
from scipy.signal import butter, filtfilt, sosfiltfilt, butter
def butter_lowpass(x, fc, fs, order=4, zero_phase=True):
    sos = butter(order, fc/(fs/2), btype='low', output='sos')
    return sosfiltfilt(sos, x) if zero_phase else sos
# Clean a 200 Hz accelerometer, keep 0-25 Hz motion band:
accel_clean = butter_lowpass(accel_z, fc=25, fs=200, order=4)
```

12 Moving Average Filter

The **Simple Moving Average (SMA)** of window M is the mean of the last M samples

— an FIR low-pass filter with equal taps $b_k = 1/M$:

$$y[n] = \frac{1}{M} \sum_{k=0}^{M-1} x[n-k].$$

The **Weighted MA (WMA)** uses unequal taps; the **Exponential MA (EMA)** weights recent samples geometrically. “Rolling mean” is the SMA computed over a sliding window.

The SMA is FIR; its frequency response is the geometric sum

$$H(e^{j\omega}) = \frac{1}{M} \sum_{k=0}^{M-1} e^{-j\omega k} = \frac{1}{M} \frac{\sin(\omega M/2)}{\sin(\omega/2)} e^{-j\omega(M-1)/2}.$$

The magnitude is a *Dirichlet (aliased sinc)* kernel: a low-pass with nulls at $\omega = 2\pi m/M$ and side-lobes. The linear-phase factor $e^{-j\omega(M-1)/2}$ means a constant delay of $(M-1)/2$ samples — easy to compensate.

The EMA obeys $y[n] = \alpha x[n] + (1 - \alpha)y[n-1]$, giving $H(z) = \frac{\alpha}{1 - (1 - \alpha)z^{-1}}$ — a single pole at $z = 1 - \alpha$. Smaller α = heavier smoothing = lower cut-off. It needs only *one* stored value, ideal for edge deployment.

Variant	Formula	Trade-off
SMA	equal weights	simple; poor stopband (side-lobes); lag $(M-1)/2$
WMA	linear weights	less lag than SMA, more compute
EMA	one-pole recursion	$O(1)$ memory, smooth; infinite (fading) memory

Applications / limits in Trinetra-AI. Cheap first-pass smoothing of any sensor, and the basis of rolling features (Phase 4 §8). *Limitation:* the SMA is a poor frequency-selective filter (leaky side-lobes) and adds lag; for real band-limiting prefer Butterworth. Use the SMA/EMA for quick, low-cost smoothing and feature computation, not for precise anti-alias or motion-band isolation.

```
import numpy as np
def sma(x, M): return np.convolve(x, np.ones(M)/M, mode='same')
def ema(x, alpha):
    y = np.empty_like(x, float); y[0] = x[0]
    for n in range(1, len(x)):
        y[n] = alpha*x[n] + (1-alpha)*y[n-1]
    return y
```

13 Savitzky–Golay Filter

The **Savitzky–Golay (SG) filter** smooths by fitting a low-degree polynomial to each sliding window of $2m+1$ points by *least squares*, then replacing the centre sample with the fitted value. Unlike a moving average, it *preserves higher moments* (peak height/width and derivatives) of the signal.

Fit degree- p polynomial $\hat{x}(i) = \sum_{j=0}^p c_j i^j$ to window points $i = -m, \dots, m$. Stack the design matrix $\mathbf{A}$ with $A_{ij} = i^j$. Least squares gives $\hat{\mathbf{c}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{x}$, and the smoothed centre value is the first fitted coefficient:

$$y[0] = \mathbf{e}_0^\top (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{x} = \sum_{k=-m}^m h_k x[k].$$

Because $\mathbf{A}$ depends only on window geometry, the weights h_k are *precomputed once* — SG is just an FIR convolution with special, moment-preserving coefficients. Using row r of $(\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top$ instead yields a smoothed r -th *derivative* directly — extremely useful for estimating clean velocity/acceleration from a noisy position/velocity signal.

	Moving average	Savitzky–Golay
Basis	local constant (mean)	local polynomial (degree p)
Preserves peaks?	no (flattens)	yes
Gives derivatives?	no	yes (clean $\dot{x}, \ddot{x}$)
Cost	minimal	small (fixed FIR)
Best for	flat/slow signals	feature-rich, peaky signals

SG shines wherever Trinetra-AI must smooth *without destroying shape*: cleaning magnetometer anomaly profiles (whose peaks *are* the localisation feature), and computing low-noise derivatives (jerk from accel, angular acceleration from gyro) for feature engineering and manoeuvre detection.

```
from scipy.signal import savgol_filter
# Smooth while preserving peaks (window 21, cubic):
mag_smooth = savgol_filter(mag_x, window_length=21, polyorder=3)
# Clean first derivative (velocity from noisy position), same call:
vel_clean = savgol_filter(pos, window_length=21, polyorder=3, deriv=1, delta=1/fs)
```

14 Advanced Signal Processing

Real navigation signals are non-stationary and transient-rich, which is exactly where the single-window FFT and static filters fall short. These advanced tools handle time-varying spectra, transients, and adaptive scenarios.

14.1 Short-Time Fourier Transform (STFT)

The **STFT** slides a window w across the signal and Fourier-transforms each segment, producing a time–frequency map:

$$X(\tau, \omega) = \sum_n x[n] w[n - \tau] e^{-j\omega n}.$$

$|X|^2$ is the **spectrogram**.

There is a fundamental resolution trade-off (a time–frequency uncertainty): a short window localises *time* well but *frequency* poorly, and vice-versa. Use the STFT to watch how vibration content evolves through a manoeuvre.

14.2 Wavelet Transform (CWT / DWT)

The **Continuous Wavelet Transform** correlates the signal with scaled/shifted copies of a mother wavelet ψ :

$$W(a, b) = \frac{1}{\sqrt{a}} \int x(t) \psi^* \left(\frac{t - b}{a} \right) dt,$$

where a is scale (inverse frequency) and b is time. The **Discrete Wavelet Transform (DWT)** uses a dyadic filter-bank (cascaded low/high-pass + downsampling) for an efficient multi-resolution decomposition.

Wavelets give *multi-resolution* analysis — good time resolution at high frequency (catching transients like impacts) and good frequency resolution at low frequency (tracking slow drift). Ideal for de-noising non-stationary IMU data (wavelet thresholding) and detecting transient faults.

14.3 Hilbert transform, cepstrum, EMD

The **Hilbert transform** builds the analytic signal $x_a(t) = x(t) + j\mathcal{H}\{x(t)\}$, from which the **envelope** $|x_a|$ and **instantaneous frequency** $\frac{1}{2\pi} \frac{d}{dt} \arg x_a$ follow — used for vibration *envelope* analysis of bearing faults.

The **cepstrum** $c[n] = \text{IDFT}\{\log|\text{DFT}\{x[n]\}|\}$ turns periodic structure in the *spectrum* (harmonic families, echoes) into isolated peaks — good for detecting repetitive mechanical faults.

Empirical Mode Decomposition (EMD) adaptively splits a non-linear, non-stationary signal into *Intrinsic Mode Functions* by iterative “sifting”; with the Hilbert transform it forms the **Hilbert–Huang Transform** for instantaneous-frequency analysis without assuming stationarity.

14.4 Adaptive & Kalman filtering (bridge to Phase 7)

Adaptive filtering (LMS/RLS, §7) tracks changing noise online. **Kalman filtering** is itself an *optimal recursive filter* — the seam between Phase 5 and Phase 7. It fuses a motion model with noisy measurements to produce a minimum-variance state estimate, and can be seen as an adaptive, model-based low-pass filter whose “cut-off” is set by the ratio of process to measurement noise (**Q** vs **R**). Everything in this document exists to give the Kalman filter clean inputs and honest noise covariances.

Tool	Where it is useful inside Trinetra-AI
STFT / spectrogram	watch vibration evolve during manoeuvres; time-varying features
Wavelet (CWT/DWT)	de-noise non-stationary IMU; catch transient faults; multi-scale features
Hilbert / envelope	bearing/rotor fault envelope; instantaneous heading-rate
Cepstrum	detect repetitive mechanical faults, echoes
EMD / HHT	decompose non-stationary sensor signals adaptively
Adaptive (LMS/RLS)	cancel time-varying rotor vibration from IMU
Kalman	the fusion core itself (Phase 7); model-based optimal filtering

15 Signal Processing Libraries

Library	Purpose / key functions	Trinetra-AI use	Limitation
NumPy	arrays, <code>fft.rfft</code> , <code>convolve</code> , linear algebra	core math everywhere	low-level only
SciPy	<code>scipy.signal</code> , <code>scipy.fft</code> , <code>interpolate</code> , <code>stats</code>	the DSP workhorse	not ML/deep
<code>scipy.signal</code>	<code>butter</code> , <code>firwin</code> , <code>filtfilt</code> , <code>welch</code> , <code>savgol_filter</code> , <code>spectrogram</code>	all filtering & PSD	manual design choices
PyWavelets	<code>cwt</code> , <code>wavedec</code> , thresholding	wavelet de-noising & features	wavelet choice matters
librosa	STFT, spectrogram, framing utilities	spectrogram features (audio-origin but general)	audio-centric API
statsmodels	ACF/PACF, ADF, decomposition	stationarity tests, noise structure	weak for multivariate DL
OpenCV	2-D transforms, image of spectrogram/map	spatial magnetic-map ops, spectrogram-as-image for CNNs	not 1-D DSP focused
FilterPy	<code>KalmanFilter</code> , common noise	bridge to Phase 7 fusion	pedagogical speed, not optimised

Recommended stack: **NumPy+SciPy** for acquisition, filtering, and spectra; **PyWavelets** for multi-resolution de-noising; **statsmodels** for noise characterisation; **FilterPy** to prototype the Kalman fusion that consumes these cleaned signals. Keep the DSP layer in SciPy (fast, battle-tested) and reserve PyTorch for the learned modules of Phase 4.

16 Research Papers

Verified, correctly attributed. Priority: ★★★ read first, ★★ core, ★ as needed.

16.1 Foundational

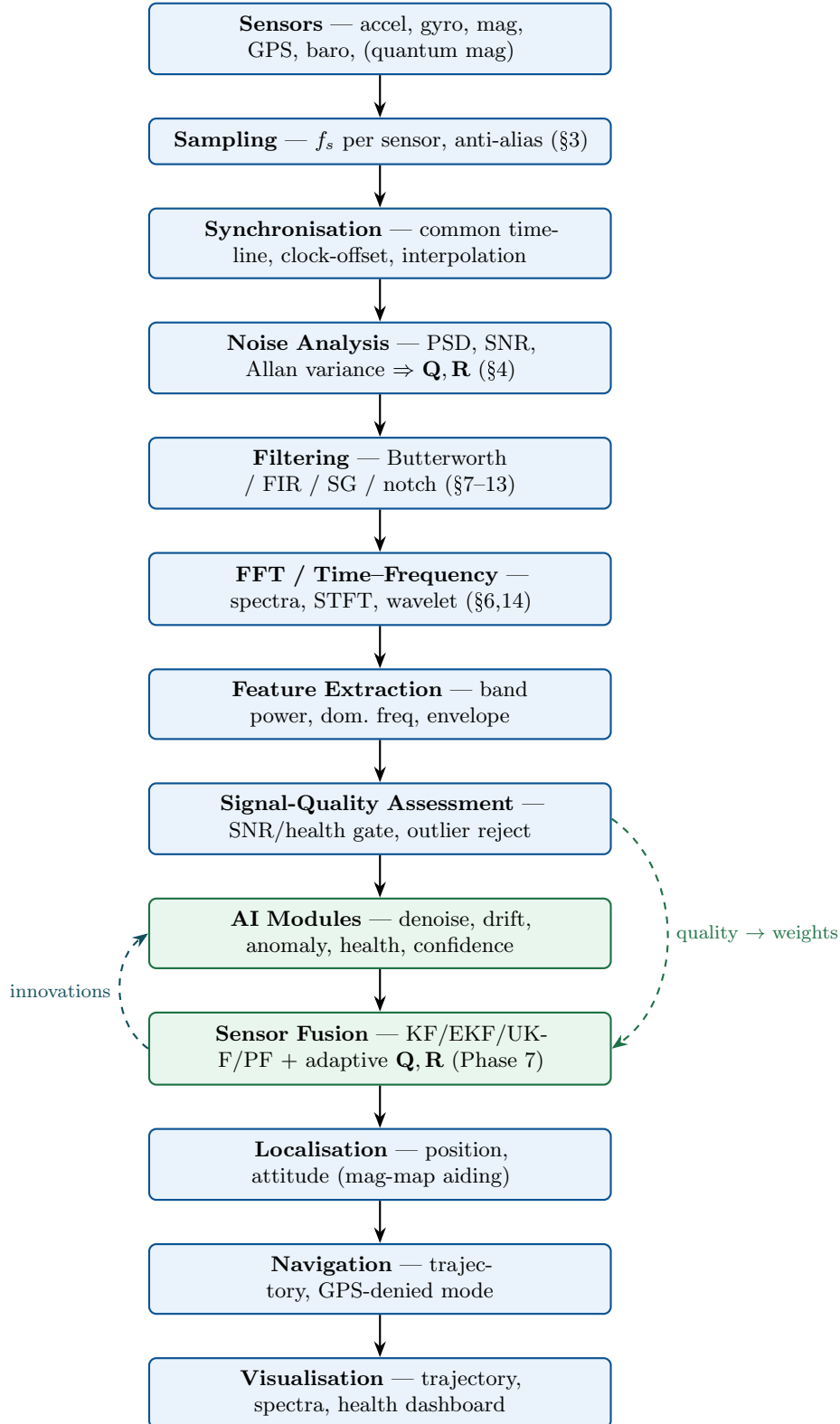
Work	Author(s), year	Contribution	Pri.
Sampling / information theory	Shannon, 1949 (Nyquist, 1928)	sampling theorem	***
An algorithm for the machine calculation of complex Fourier series (FFT)	Cooley & Tukey, 1965	$O(N \log N)$ FFT	***
On the theory of filter amplifiers (Butterworth)	Butterworth, 1930	maximally-flat filter	**
Smoothing and differentiation of data by least-squares procedures	Savitzky & Golay, 1964	SG smoothing filter	**
The use of FFT for estimation of power spectra (Welch method)	Welch, 1967	averaged PSD	**
A new approach to linear filtering and prediction problems	Kalman, 1960	the Kalman filter	***
Statistics of atomic frequency standards (Allan variance)	Allan, 1966	noise-identification metric	**
A theory for multiresolution signal decomposition (wavelets)	Mallat, 1989	DWT filter-bank	*
The empirical mode decomposition... (Hilbert–Huang)	Huang et al., 1998	EMD/HHT for non-stationary signals	*
Adaptive switching circuits (LMS)	Widrow & Hoff, 1960	adaptive filtering	*

16.2 Recent & navigation-specific (DSP × learning)

Paper	Author(s), year	How it helps Trinetra-AI	Pri.
Denoising IMU Gyroscopes with Deep Learning for Open-Loop Attitude Estimation (IEEE RA-L 5(3):4796–4803)	Brossard, Bonnabel & Barrau, 2020	dilated-CNN gyro de-noising; the exact DSP→attitude idea; EuRoC/TUM-VI	***
Estimation of IMU/MARG orientation using a gradient-descent algorithm (Madgwick filter, ICORR)	Madgwick, Harrison & Vaidyanathan, 2011	lightweight complementary-style orientation filter	**
AI-IMU Dead-Reckoning (IEEE T-IV)	Brossard, Barrau & Bonnabel, 2020	learning-aided inertial dead-reckoning	**
MEMS-IMU RNN de-noising line (e.g. Jiang et al. 2018/2019; Han et al. 2021)	various, 2018–2021	recurrent de-noising baselines for MEMS drift	*

Reading path: Shannon → Cooley–Tukey → Butterworth/Savitzky–Golay give the classical foundation; Kalman (1960) is the seam to Phase 7; Brossard et al. (2020) shows the modern DSP-meets-deep-learning pattern Trinetra-AI extends. **Citation hygiene:** confirm every DOI/venue on the official page before it enters a report — fabricated references are the fastest way to lose reviewer trust.

17 Signal Processing Pipeline inside Trinetra-AI



Block detail. *Sampling* fixes rate and anti-aliasing; *synchronisation* puts all sensors on one clock; *noise analysis* converts static recordings into the covariances the filter needs; *filtering* band-limits and de-noises; *FFT/TF* exposes spectral structure; *feature extraction* and *quality assessment* feed both the AI modules and the fusion weights; the dashed loops show DSP-derived signal-quality tuning the fusion forward, and filter innovations feeding anomaly/health backward.

18 Practical Implementation

Notebook	Contents	Expected output
01_sampling_sync.ipynb	load raw sensors, resample, clock-offset	aligned multi-rate frame
02_noise_psd_allan.ipynb	PSD (Welch), SNR, Allan variance	noise budget, $\mathbf{Q}$, $\mathbf{R}$ values
03_fft_spectra.ipynb	FFT, windowing, spectrogram	spectra & vibration tones
04_filters.ipynb	Butterworth/FIR/SG/notch, <code>filtfilt</code>	filtered signals + Bode plots
05_wavelet_denoise.ipynb	DWT thresholding, EMD	de-noised non-stationary signals
06_features.ipynb	band power, envelope, SG-derivatives	feature table for Phase-4 ML
07_quality_gate.ipynb	SNR/health scoring, outlier reject	per-sample quality flags

Evaluation metrics. SNR improvement (dB), RMSE vs a clean/simulated reference, attenuation at target frequency (dB), phase delay (samples), and — end-to-end — reduction in downstream drift/ATE. **Visualisation:** raw-vs-filtered overlays, magnitude/phase (Bode) plots, spectrograms, PSD, pole-zero plots.

Common mistakes: filtering *after* aliasing has already occurred; using a causal filter then forgetting its phase delay in fusion timing; cut-off so low it eats real dynamics; applying a global FFT to non-stationary manoeuvre data; estimating $\mathbf{R}$ from a *moving* (non-static) recording. **Best practices:** anti-alias in analog before the ADC; prefer `filtfilt`/`sosfiltfilt` (zero-phase) offline; choose cut-offs from the measured motion band; characterise noise on static data; version filter configs; always keep a raw copy for audit.

19 Project Mapping

DSP topic	Problem it solves	AI module served	Fusion / quantum link
Sampling/sync	align multi-rate sensors	all (clean inputs)	every KF update; quantum mag. channel
Noise analysis	set $\mathbf{Q}$, $\mathbf{R}$ honestly	confidence scoring	KF/EKF/UKF gain; shot-noise model
Low-pass / Butterworth	remove vibration	denoise, drift	KF measurement conditioning
High-pass	remove drift/DC	drift detection	complementary filter
Band-pass / notch	isolate/reject a band	fault detection	reject rotor tone before fusion
FFT / STFT	expose spectra	anomaly, health	vibration gating; magnetic anomaly
Wavelet / EMD	non-stationary de-noise	anomaly, denoise	transient rejection pre-fusion
Savitzky–Golay	smooth + derivatives	feature engineering	clean mag-anomaly for map-match
Adaptive Kalman	/ track & fuse	uncertainty weights	→ the fusion core itself (Phase 7)

20 Interview & Research Preparation

20.1 40 interview questions

1. Define DSP and why sampled+quantised matters.
2. State the sampling theorem.

3. What is aliasing; why can't a digital filter fix it?
4. Derive the folding frequency of an aliased tone.
5. Why anti-alias in analog before the ADC?
6. Define Nyquist frequency.
7. Quantisation noise variance and SQNR-per-bit.
8. Energy vs power signal — sensor examples.
9. Stationary vs non-stationary; test?
10. Why can't a moving IMU use a single global FFT?
11. White vs Gaussian vs coloured noise.
12. Define SNR and noise density (ARW).
13. Derive white-noise $\rightarrow$ random-walk variance growth.
14. Why does accel bias give t^2 position error?
15. What is a PSD; Wiener–Khinchin.
16. Spectral leakage and windowing.
17. DFT vs FFT complexity.
18. Sketch the radix-2 butterfly.
19. FIR vs IIR: stability & phase.
20. What is linear phase and why prized in navigation?
21. Transfer function of an FIR filter.
22. Pole condition for IIR stability.
23. Low-pass first-order $H(s)$ and roll-off.
24. High-pass use in the complementary filter.
25. Band-pass vs notch.
26. Why is Butterworth “maximally flat”?
27. Its -3 dB point vs order.
28. Butterworth vs Chebyshev vs Bessel vs Elliptic.
29. SMA frequency response (Dirichlet kernel).
30. EMA as one-pole IIR.
31. Savitzky–Golay vs moving average.
32. How SG yields clean derivatives.
33. STFT time–frequency trade-off.
34. CWT vs DWT.
35. Hilbert envelope for fault detection.
36. Cepstrum use.
37. EMD/HHT for non-stationary signals.
38. LMS update rule.
39. Kalman filter as a model-based filter.
40. Why DSP precedes fusion.

20.2 25 viva questions

1. Walk your DSP pipeline end to end.
2. Justify each sensor's sampling rate.
3. How do you set $\mathbf{Q}$, $\mathbf{R}$ from data?
4. Defend your filter-family choice per sensor.
5. Show the aliasing spectrum-replication picture.
6. Why zero-phase filtering offline?
7. Explain the delay a causal filter adds and how fusion compensates.
8. When wavelet over FFT?
9. How does SG preserve peaks mathematically?
10. Derive Butterworth poles.
11. Explain Allan variance slopes (ARW, bias instability, RRW).
12. How would a quantum magnetometer change your noise model?
13. Where does DSP feed the AI modules?
14. Complementary filter: which band from which sensor?
15. How do you detect a faulty sensor spectrally?
16. SMA vs Butterworth trade-off.
17. What breaks if sensors are unsynchronised?
18. Why is GPS noise non-Gaussian and why does it matter?
19. Show FFT $\rightarrow$ feature $\rightarrow$ model flow.
20. Validate a filter (metrics/plots).
21. EMA vs SMA memory/lag.
22. Why maximally-flat passband for motion data?
23. Notch design for a shifting rotor tone.
24. How does DSP bound integration drift?
25. Kalman as the seam to Phase 7 — explain.

20.3 20 research questions

1. Learned vs classical de-noising: measurable drift reduction?
2. Optimal cut-off vs downstream ATE?
3. Wavelet-threshold vs Butterworth for non-stationary IMU?
4. Adaptive notch tracking of throttle-dependent rotor tones.
5. Can network uncertainty replace hand-tuned $\mathbf{R}$?
6. Quantum-magnetometer shot-noise model for fusion.
7. Allan-variance-informed $\mathbf{Q}$ vs empirical $\mathbf{Q}$.
8. Spectrogram-CNN fault detection accuracy.
9. EMD vs DWT for transient isolation.

10. Phase-distortion impact of causal IIR on fusion.
11. Joint DSP+fusion end-to-end optimisation.
12. Robustness of mag-anomaly features to environment change.
13. Online SNR gating of sensor weights — stability?
14. Minimal filter set for edge (Jetson) real-time.
15. Cross-sensor coherence for sync-error estimation.
16. Learned anti-alias/decimation front-ends.
17. Cepstral features for bearing-fault prognosis.
18. Effect of window taper on weak-fault detectability.
19. Hilbert instantaneous-frequency for manoeuvre segmentation.
20. DSP pre-processing effect on learned drift-prediction generalisation.

20.4 15 coding questions

1. Implement a radix-2 FFT and verify against `numpy.fft`.
2. Design a 4th-order Butterworth LPF and plot its Bode response.
3. Zero-phase filter a signal with `filtfilt`; measure delay vs causal.
4. Compute a Welch PSD and mark the noise floor.
5. Build an SMA and an EMA; compare step responses.
6. Implement Savitzky–Golay from the least-squares definition.
7. Detect the dominant frequency of a noisy sinusoid.
8. Simulate aliasing by under-sampling a tone.
9. Estimate SNR before/after filtering.
10. Compute Allan variance from a static gyro log.
11. Wavelet-threshold de-noise a transient signal.
12. Design a notch filter at a given rotor tone.
13. Build a spectrogram and label a manoeuvre.
14. Resample and synchronise two sensors to one clock.
15. Extract band-power features over sliding windows.

21 Glossary, Summary & Roadmap

21.1 Glossary

Aliasing — high frequencies folding below f_N due to under-sampling. **ADC** — analog-to-digital converter (samples + quantises). **Allan variance** — metric separating ARW, bias instability, rate random walk. **ARW** — angle random walk; gyro noise whose angle error grows as $\sqrt{t}$. **Bode plot** — magnitude & phase vs frequency. **Cut-off** (f_c) — -3 dB band edge of a filter. **DFT/FFT** — discrete Fourier transform / its $O(N \log N)$ algorithm. **FIR/IIR** — non-recursive (finite) / recursive (infinite) impulse response filter. **Linear phase** — constant group delay; preserves waveform shape. **Nyquist freq.** (f_N) — $f_s/2$. **PSD** — power spectral density (power

per Hz). **Quantisation noise** — rounding error, variance $\Delta^2/12$. **SNR** — signal-to-noise ratio. **STFT / spectrogram** — windowed FFT over time / its squared magnitude. **Wavelet** — basis giving multi-resolution time–frequency analysis. **Zero-phase filtering** — forward+backward filtering (`filtfilt`) with no net delay.

21.2 Summary

DSP is the layer that makes raw sensor data trustworthy. We classify signals, sample them without aliasing, synchronise them, *characterise* their noise (turning static recordings into **Q, R**), then *filter* them (Butterworth as the default, FIR for linear phase, Savitzky–Golay to preserve peaks, notch/adaptive for tones, wavelet/EMD for non-stationary transients), and finally *transform* them (FFT/STFT) into features and quality signals. The output feeds both the AI modules (Phase 4) and the Kalman fusion core (Phase 7). The recurring theme: navigation *integrates* sensors, integration amplifies low-frequency error, so disciplined DSP up front is what bounds drift downstream.

21.3 Roadmap: how Phase 5 connects to Phases 6 & 7

Phase 5 (this) touchpoint	Phase 6 — Sensors & IMU	Phase 7 — Fusion & Kalman
Noise analysis, Allan variance	sensor error models, bias/scale, calibration	sets Q, R for KF/EKF/UKF
Anti-alias & sampling	datasheet rates, ADC specs	measurement update cadence
Filtering (LP/HP/BP/SG)	de-noised, calibrated sensor outputs	clean measurements the filter assumes
FFT / features / quality	health & fault signatures	adaptive weighting, outlier gating
Complementary / adaptive / Kalman intro	orientation from accel+gyro+mag	full state estimation & sensor fusion

Enter Phase 6 knowing *why* each sensor is filtered and *how* its noise is measured; enter Phase 7 with clean signals and honest covariances in hand. DSP is the foundation that makes the Kalman filter’s optimality assumptions approximately true — without it, fusion is built on sand.

End of Phase 5 notes. Next: Phase 6 (Sensors & IMU Fundamentals) → Phase 7 (Sensor Fusion & Kalman Filters).